



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ \_\_\_\_\_ «Информатика и системы управления»

КАФЕДРА \_\_\_\_\_ «Теоретическая информатика и компьютерные технологии»

**Лабораторная работа № 5**  
**по курсу «Языки и методы программирования»**  
**«Монады в языке Java»**

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

*Москва 2026*

# 1 Цель

Целью данной работы является приобретение навыков использования монад `Optional` и `Stream` в программах на языке Java.

## 2 Персональный вариант

Множество интервалов вида  $[a, b]$ , где  $a$  и  $b$  – целые числа, с операциями:

1. порождение потока, содержащего интервалы множества, включающие в себя корни указанного квадратного уравнения;
2. поиск максимального по размеру интервала, включающего в себя корни указанного квадратного уравнения.

Проверить работу первой операции нужно путём группировки интервалов на 10 групп по размеру интервалов: от 0 до 1, от 1 до 2, ..., от 9 до бесконечности.

### 3 Практическая реализация

класс Interval

```
import java.util.Objects;
public class Interval {
    private final int a;
    private final int b;
    public Interval(int a, int b) {
        this.a = Math.min(a, b);
        this.b = Math.max(a, b);
    }
    public int getA() {
        return a;
    }
    public int getB() {
        return b;
    }
    public int size() {
        return b - a;
    }
    public boolean contains(double x) {
        return a <= x && x <= b;
    }
    @Override
    public String toString() {
        return "[" + a + ", " + b + "]";
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Interval)) return false;
        Interval interval = (Interval) o;
        return a == interval.a && b == interval.b;
    }
    @Override
    public int hashCode() {
        return Objects.hash(a, b);
    }
}
```

Класс IntervalSet

```
import java.util.*;
import java.util.stream.Stream;

public class IntervalSet {
    private final Set<Interval> intervals;
```

```

public IntervalSet() {
    this.intervals = new HashSet<>();
}

public void add(Interval interval) {
    intervals.add(interval);
}

public void add(int a, int b) {
    intervals.add(new Interval(a, b));
}

public Stream<Interval> intervalsContainingRoots(double a, double b,
double c) {
    List<Double> roots = solveQuadratic(a, b, c);
    if (roots.isEmpty()) {
        return Stream.empty();
    }
    return intervals.stream()
        .filter(interval -> roots.stream().anyMatch(interval::
            contains));
}

public Optional<Interval> largestIntervalContainingRoots(double a, double
b, double c) {
    List<Double> roots = solveQuadratic(a, b, c);
    if (roots.isEmpty()) {
        return Optional.empty();
    }
    return intervals.stream()
        .filter(interval -> roots.stream().anyMatch(interval::
            contains))
        .max(Comparator.comparingInt(Interval::size));
}

private List<Double> solveQuadratic(double a, double b, double c) {
    if (a == 0) {
        return (b != 0) ? Collections.singletonList(-c / b) : Collections
            .emptyList();
    }
    double discriminant = b * b - 4 * a * c;
    if (discriminant < 0) {
        return Collections.emptyList();
    } else if (discriminant == 0) {
        return Collections.singletonList(-b / (2 * a));
    }
}

```

```

    } else {
        double sqrtD = Math.sqrt(discriminant);
        return Arrays.asList(
            (-b - sqrtD) / (2 * a),
            (-b + sqrtD) / (2 * a)
        );
    }
}
}

```

## Тест классов

```

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Test {
    public static void main(String[] args) {
        IntervalSet set = new IntervalSet();

        set.add(2, 2);
        set.add(2, 3);
        set.add(2, 4);
        set.add(1, 4);
        set.add(2, 6);
        set.add(0, 5);
        set.add(2, 8);
        set.add(1, 7);
        set.add(-5, 3);
        set.add(-10, 10);
        set.add(-100, 100);
        set.add(10, 20); //
        set.add(50, 60);

        double A = 1.0;
        double B = -5.0;
        double C = 6.0;

        System.out.println("
                                : " + A + "x^2 + " + B + "x + "
                                + C + " = 0");
        System.out.println("
                                : 2.0      3.0\n");

        System.out.println(" ---
                                (10      ) ---");
        Map<Integer, List<Interval>> grouped = set.intervalsContainingRoots(A, B, C)
            .collect(Collectors.groupingBy(
                interval -> Math.min(interval.size(), 9)
            ));
    }
}

```

```

    ));

    grouped.entrySet().stream()
        .sorted(Map.Entry.comparingByKey())
        .forEach(entry -> {
            String range = (entry.getKey() == 9) ? "[9,    )" : "[" +
                entry.getKey() + ", " + (entry.getKey() + 1) + ")";
            System.out.println("                " + range +
                ": " + entry.getValue());
        });

    System.out.println();
    System.out.println(" ---
                                ---");

    set.largestIntervalContainingRoots(A, B, C)
        .ifPresentOrElse(
            interval -> System.out.println("                : " +
                interval + " (                : " + interval.size() +
                ")"),
            () -> System.out.println("
                                ")
        );

    }
}

```

## 4 Результаты

В результате работы программы получился следующий вывод:

Уравнение:  $1.0x^2 + -5.0x + 6.0 = 0$

Корни: 2.03.0

Группировка интервалов с корнями по размеру (10 групп)

Группа размер[0, 1) : [[2, 2]]

Группа размер[1, 2) : [[2, 3]]

Группа размер[2, 3) : [[2, 4]]

Группа размер[3, 4) : [[1, 4]]

Группа размер[4, 5) : [[2, 6]]

Группа размер[5, 6) : [[0, 5]]

Группа размер[6, 7) : [[2, 8], [1, 7]]

Группа размер[8, 9) :  $[[−5, 3]]$

Группа размер [9, ) :  $[[−10, 10], [−100, 100]]$

— Поиск наибольшего интервала с корнями — Найден:  $[-100, 100]$  (размер: 200)

## 5 Выводы

Реализована логика работы с множеством интервалов: фильтрация по наличию корней квадратного уравнения и поиск наибольшего подходящего интервала. Работоспособность подтверждена успешной группировкой результатов по диапазонам длин интервалов, что демонстрирует корректность вычислений и эффективность использования потоков Java.